

Polynomial Regression — Notes

1. What is Polynomial Regression?

Polynomial Regression is used when the relationship between **input (X)** and **output (y)** is **curved/non-linear** rather than a straight line.

Normal Linear Regression:

Polynomial Regression:

2. Main Idea

Instead of changing Linear Regression itself, we **create new features** from the existing feature.

For example:

Original:

X

Polynomial features:

X

X^2

X^3

Then Linear Regression uses these new features:

X, X^2 , X^3

↓

Linear Regression

↓

Curved prediction

So even though the final model is technically **linear in its parameters**, it can produce a **non-linear curve** with respect to X.

3. Example

Suppose the actual relationship looks like:



A straight line won't fit it well.

Polynomial Regression can create a curve that follows the pattern much better.

4. PolynomialFeatures

In Scikit-Learn:

```
from sklearn.preprocessing import PolynomialFeatures
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X)
```

If:

$X = [2, 3, 4]$ degree = 2 creates

features based on:

1, X , X^2

So:

2 → 1, 2, 4

3 → 1, 3, 9 4 → 1, 4, 16

Then:

```
model.fit(X_poly, y)
```

5. Choosing Degree

The **degree** controls how complex the curve can be.

Degree 1 → straight line

Degree 2 → simple curve

Degree 3 → more flexible curve

Degree 10 → very complex curve

Underfitting

Degree is too low → model is too simple → misses important patterns.

Overfitting

Degree is too high → model becomes too complex → learns **noise** instead of the real pattern.

6. Ridge Regression using Gradient Descent — Notes

This is basically combining **two concepts you already learned**:

Ridge Regression + Gradient Descent

1. Normal Linear Regression

The model tries to minimize prediction error:

Gradient Descent updates the weights:

2. Ridge Regression

Ridge adds a penalty for large weights:

So now Gradient Descent must minimize **both**:

Prediction error

+

Penalty for large weights

Therefore, the gradient contains **two parts**:

Gradient =

gradient of MSE

+

gradient of Ridge penalty

3. Weight Update

Conceptually:

Where:

- $w \rightarrow$ model coefficients
- $\eta \rightarrow$ learning rate
- $\lambda \rightarrow$ regularization strength

The Ridge part pushes large coefficients **toward zero**.

4. Coding Idea

A simplified implementation looks like:

```
weights = np.zeros(X.shape[1])
```

```
for i in range(epochs):
```

```
    y_pred = X @ weights
```

$\text{error} = y - y_{\text{pred}}$

$\text{gradient} = -(2/n) * (X.T @ \text{error})$

$\text{ridge_gradient} = 2 * \alpha * \text{weights}$

$\text{gradient} = \text{gradient} + \text{ridge_gradient}$

$\text{weights} = \text{weights} - \text{learning_rate} * \text{gradient}$

The important part is:

$\text{ridge_gradient} = 2 * \alpha * \text{weights}$

This is the **regularization component**.

Then:

$\text{gradient} = \text{gradient} + \text{ridge_gradient}$

combines:

normal Linear Regression gradient

+

Ridge penalty gradient

5. Why use Gradient Descent?

You already learned that Ridge has a closed-form solution:

But with very large datasets/features, repeatedly doing matrix operations and inversion can become expensive.

Gradient Descent instead does:

Start with weights

↓

Calculate prediction

↓

Calculate error

↓

Calculate gradient



Add Ridge penalty



Update weights



Repeat